

《ALPHAEDIT: NULL-SPACE CONSTRAINED KNOWLEDGE EDITING FOR LANGUAGE MODELS》详细的介绍

背景与问题

[cite_start]大型语言模型（LLMs）在预训练阶段学习并存储了海量的知识，并能在推理时调用这些知识 [cite: 8][cite_start]。然而，它们常常会出现“幻觉”，即生成不正确或过时的信息 [cite: 9][cite_start]。虽然可以通过微调（Fine-Tuning）来更新知识，但这种方法通常非常耗时 [cite: 10]。

[cite_start]为了解决这个问题，**模型编辑（Model Editing）** 技术应运而生。它旨在有针对性地更新模型中的目标知识，同时保护其它未修改的知识不被破坏 [cite: 2, 11]。

[cite_start]目前，一类主流的模型编辑方法遵循“**定位-再编辑**”（locate-then-edit）的范式 [cite: 3, 14]。其核心思想是：

1. [cite_start]**定位**：首先通过因果追踪等方法，找到与特定知识相关的影响力较大的模型参数（例如前馈网络 FFN 中的权重矩阵 W ） [cite: 15]。
2. [cite_start]**编辑**：然后通过引入一个扰动 Δ 来修改这些参数，从而更新知识 [cite: 3, 15]。

现有方法存在的核心问题：

[cite_start]现有方法在计算扰动 Δ 时，通常需要在目标函数中平衡两个相互冲突的目标：一个是最小化“待更新知识”的输出误差（ e_1 ），另一个是最小化“需保留知识”的输出误差（ e_0 ） [cite: 17, 18] [cite_start]。为了确保更新成功，研究人员往往会优先考虑最小化 e_1 [cite: 19]。

这种做法带来了严重的负面影响：

- [cite_start]**过拟合与表征漂移**：模型会过拟合于待更新的知识，导致模型内部的隐藏层表征分布发生偏移 [cite: 30, 31]。
- [cite_start]**模型遗忘与崩溃**：在进行多次连续编辑（sequential editing）时，这种过拟合会不断累积，最终侵蚀模型保留原有知识和生成连贯句子的能力，导致**模型遗忘**（Forgetting）甚至**模型崩溃**（Collapse），即模型开始生成无意义的重复性文本 [cite: 21, 32]。

为了应对上述挑战，该论文提出了一种名为 **AlphaEdit** 的新型模型编辑方法。

2.1 自回归语言模型（AUTOREGRESSIVE LANGUAGE MODEL）解析

文档中2.1节主要介绍了自回归大型语言模型（LLM）的核心工作机制，尤其是隐藏状态的计算方式及前馈网络（FFN）层在知识存储中的作用，具体内容如下：

1. 自回归模型的核心功能

自回归大型语言模型（LLM）的核心能力是基于序列中已有的前序 tokens 预测下一个 token x 。这种预测依赖于模型各层计算的隐藏状态（hidden state），而隐藏状态的更新是模型理解和生成文本的关键。

2. 隐藏状态的计算公式

第 l 层的隐藏状态 h^l 由三部分组成，公式如下：

$$h^l = h^{l-1} + a^l + m^l$$

其中：

- h^{l-1} 是上一层（ $l-1$ 层）的隐藏状态；
- a^l 是当前层注意力模块（attention block）的输出；
- m^l 是当前层前馈网络（FFN）的输出，其计算公式为：
$$m^l = W_{out}^l \sigma(W_{in}^l \gamma(h^{l-1} + a^l))$$

这里, W_{in}^l 和 W_{out}^l 是 FFN 层的权重矩阵, σ 是非线性激活函数 (如 ReLU), γ 表示层归一化 (layer normalization)。

3. FFN层的知识存储作用

文档特别指出, FFN 层中的输出权重矩阵 W_{out}^l 可被理解作为一种“线性联想记忆” (linear associative memory), 起到键-值 (key-value) 存储的作用, 用于信息检索。

- 具体而言, LLM 中存储的知识可形式化为三元组 (s, r, o) (主体 s 、关系 r 、客体 o), 例如 “(最新奥运会, 举办地是, 巴黎)”。
- W_{out}^l 的作用是将编码 (s, r) 的输入键 (key) k 与编码 o 的对应值 (value) v 关联起来,

即:

$$\underbrace{m^l}_v = W_{out}^l \underbrace{\sigma(W_{in}^l \gamma(h^{l-1} + a^l))}_k$$

其中, k 是经过 FFN 输入层和激活函数处理后的键向量, v 是 FFN 的输出 (即 m^l), 代表与 k 对应的 value。

4. 对模型编辑的启发

由于 W_{out}^l 承担知识的键-值关联存储功能, 多数模型编辑方法 (包括本文提出的 AlphaEdit) 选择修改 FFN 层来实现知识更新。为简化表述, 后续章节中用 W 指代 W_{out}^l 。

综上, 2.1节通过公式和实例阐明了自回归LLM的隐藏状态计算逻辑, 并重点强调了FFN层 (尤其是 W_{out}^l) 在知识存储中的核心作用, 为后续模型编辑方法 (通过修改FFN层实现知识更新) 奠定了理论基础。

AlphaEdit 方法详解

[cite_start]AlphaEdit 的核心思想是: 在将扰动 Δ 应用于模型参数之前, 先将其投影到“需保留知识”的 **零空间 (Null Space)** 上 [cite: 5][cite_start]。通过这种方式, 可以从理论上保证编辑后的模型在被问及需保留的知识时, 其输出保持不变, 从而彻底消除了对原有知识的破坏 [cite: 6]。

1. 核心概念: 零空间 (Null Space)

- [cite_start]**定义**: 在线性代数中, 给定一个矩阵 A , 其左零空间是指所有满足 $BA = 0$ 的矩阵 B 的集合 [cite: 70]。
- 在模型编辑中的应用**:
 - 论文将“需保留知识”的输入键 (keys) 堆叠成一个矩阵 K_0 。
 - [cite_start]如果施加的扰动 Δ' 位于 K_0 的零空间中, 那么根据定义有 $\Delta'K_0 = 0$ [cite: 71]。
 - 此时, 编辑后的权重矩阵 $(W + \Delta')$ 在作用于保留知识的键 K_0 时, 其输出为:
 $(W + \Delta')K_0 = WK_0 + \Delta'K_0 = WK_0 = V_0$
[cite_start]这里的 V_0 是保留知识对应的原始值 (values) [cite: 71]。
 - [cite_start]这个公式从理论上证明了, 只要扰动位于 K_0 的零空间内, 就不会影响模型对保留知识的键值对关联, 从而确保了这部分知识的完整性 [cite: 71, 72]。

[cite_start]根据您提供的论文, 这个代表“需保留知识”的输入键 (keys) 的矩阵, 用符号 K_0 表示, 它 **不是模型权重**, 而是**通过对人为输入的大量文本进行编码得到的矩阵** [cite: 62, 63]。

以下是具体的解释:

- [cite_start]**来源和构成**: 该矩阵由大量从外部知识源 (如维基百科) 中抽取的知识三元组 ((s, r, o) - 主语, 关系, 宾语) 编码而成 [cite: 63][cite_start]。在论文的实践中, 研究人员通常会随机选择约100,000个这样的三元组来构建 K_0 [cite: 63]。
- 编码过程**:

1. 对于每一个知识三元组，例如（“美国总统”，“是”，“拜登”），模型会处理一个相关的提示（prompt），比如“The President of the United States is”。
 2. [cite_start]当这个提示流经模型时，在被选定要编辑的前馈网络（FFN）层，其输入会被编码成一个“键”向量 $\mathbf{k}$ [cite: 48, 49]。
 3. 论文将这100,000个知识样本对应的“键”向量 $\mathbf{k}$ 横向堆叠起来，形成一个巨大的矩阵 $K_0 = [\mathbf{k}_1 | \mathbf{k}_2 | \dots | \mathbf{k}_{100,000}]$ [cite: 56, 63]。
- [cite_start]**作用**：这个 K_0 矩阵在数学上代表了模型中存储的、不希望在本次编辑中被修改的“背景知识”或“保留知识”的输入部分 [cite: 59, 62][cite_start]。通过将其用于零空间投影，AlphaEdit方法可以确保对模型参数的修改不会影响模型处理这些背景知识时的输出 [cite: 71, 72]。

总结来说， K_0 是一个根据海量外部文本数据动态生成的数据矩阵，而非模型固有的、预先存在的权重矩阵。它为模型编辑提供了一个关于“什么不该被改变”的数学约束。

2. 零空间投影的实现

[cite_start]直接计算高维矩阵 K_0 （通常包含数十万个知识样本）的零空间在计算和存储上都极具挑战性 [cite: 76, 77]。

- [cite_start]**简化计算**：论文采用了一个技巧，即使用非中心协方差矩阵 $K_0 K_0^T$ 的零空间来代替 K_0 的零空间 [cite: 78][cite_start]。这两个矩阵拥有相同的零空间（详见论文附录B.2的证明），但 $K_0 K_0^T$ 的维度是 $d_0 \times d_0$ （ d_0 是FFN中间层的维度），远小于 K_0 的列数（100,000），从而大大降低了计算复杂度 [cite: 78]。
- **投影矩阵的构建**：
 1. 对 $K_0 K_0^T$ 进行奇异值分解（SVD）：
 $\{U, \Lambda, U^T\} = \text{SVD}(K_0 K_0^T)$
[cite_start]其中 U 是由 $K_0 K_0^T$ 的特征向量组成的矩阵 [cite: 79]。
 2. [cite_start]移除 U 中与非零特征值（在实践中是大于一个阈值如 10^{-2} 的特征值）对应的特征向量，将剩余的特征向量构成子矩阵 $\hat{U}$ [cite: 84]。
 3. 构建投影矩阵 P ：
 $P = \hat{U}(\hat{U})^T$
[cite_start]这个矩阵 P 可以将任意扰动 Δ 的列向量映射到 $K_0 K_0^T$ 的零空间中 [cite: 80]。

3. AlphaEdit 的优化目标与求解

AlphaEdit 基于零空间投影，对传统的编辑目标函数进行了优化。

- **传统方法的序贯编辑目标**：以 MEMIT 方法为例，其目标函数如下：

$$\Delta_{MEMIT} = RK_1^T (K_p K_p^T + K_1 K_1^T + K_0 K_0^T)^{-1}$$

- $R = V_1 - WK_1$ ：表示当前待更新知识的残差向量。
- K_1, V_1 ：当前待更新知识的键和值矩阵。
- K_p, V_p ：之前已经更新过的知识的键和值矩阵。
- K_0, V_0 ：需要保留的原始知识的键和值矩阵。

[cite_start]这个公式试图同时平衡对 K_1 的更新以及对 K_p 和 K_0 的保留 [cite: 101]。

- **AlphaEdit 的序贯编辑目标**：AlphaEdit 的优化过程如下：

1. [cite_start]将原始扰动 $\tilde{\Delta}$ 替换为投影后的扰动 $\tilde{\Delta}P$ ，这保证了扰动不会影响到由 K_0 代表的保留知识 [cite: 88]。
2. [cite_start]由于步骤 (1) 已经保护了保留知识，因此可以从目标函数中移除与 K_0 相关的项 [cite: 89]。
3. [cite_start]引入正则化项 $\|\tilde{\Delta}P\|^2$ 保证收敛的稳定性 [cite: 90]。

经过优化，其针对序贯编辑的最终目标函数变为：

$$\Delta = \arg \min_{\tilde{\Delta}} (\|(W + \tilde{\Delta}P)K_1 - V_1\|^2 + \|\tilde{\Delta}P\|^2 + \|\tilde{\Delta}PK_p\|^2)$$

[cite_start]这个目标函数只关注**更新知识** (K_1) 和**维持之前编辑过的知识** (K_p)，而无需再考虑保留原始知识 (K_0) [cite: 96]。

- **AlphaEdit 的最终解**：通过正规方程 (Normal Equation) 求解上述目标函数，可以得到最终施加于模型参数的扰动 $\Delta_{AlphaEdit}$ ：

$$\Delta_{AlphaEdit} = RK_1^T P (K_p K_p^T P + K_1 K_1^T P + I)^{-1}$$

- I 是单位矩阵。
- [cite_start]与 Δ_{MEMIT} 相比，AlphaEdit 的解只是在关键项中加入了投影矩阵 P [cite: 102] [cite_start]。这一微小的修改使得 AlphaEdit 可以轻松集成到大多数现有的模型编辑算法中，作为一个即插即用的增强模块 [cite: 39, 102]。

[cite_start]**总结**：AlphaEdit 通过一个巧妙的零空间投影操作，将模型编辑中“更新知识”和“保留知识”这两个相互冲突的目标解耦。它允许模型在更新时可以完全专注于最小化新知识的误差，而无需担心破坏原有的海量知识，从而有效缓解了现有方法在序贯编辑中出现的模型遗忘和崩溃问题 [cite: 36]。

识别和定位模型特定层

[cite_start]在您提供的论文 "Alphaedit" 中，实现定位的方法叫做**因果追踪 (causal tracing)**。这是一种用来识别和定位模型中哪些特定层和神经元与特定知识的存储和回忆最相关的技术 [cite: 372]。

其具体实现步骤如下：

1. [cite_start]**注入噪声**：首先，研究人员会将一个包含特定事实的提示（例如，“埃菲尔铁塔位于”）输入到模型中，并在这个过程中向模型各层的隐藏状态（hidden states）中注入高斯噪声 [cite: 373]。这个噪声会干扰模型的正常计算，导致模型无法给出正确的答案。
2. [cite_start]**逐步恢复**：接下来，他们会逐层、逐步地将加了噪声的隐藏状态恢复到其原始的、无噪声的值 [cite: 373]。
3. [cite_start]**分析影响**：在恢复过程中，研究人员会持续观察模型的最终输出。如果恢复某一特定层的隐藏状态能够显著地让模型的输出恢复到正确的答案（例如，从混乱的输出变为“巴黎”），那么就证明该层对于这个特定知识的存储和处理至关重要 [cite: 374]。

[cite_start]通过这种方式，研究人员可以像进行“因果实验”一样，精准地“追踪”到对特定事实知识影响最大（即因果关系最强）的**影响力层 (influential layers)**，从而将它们 pinpoint (精确定位) 为后续编辑的目标 [cite: 372, 374]。

K_0 矩阵

根据您提供的文件， K_0 矩阵是通过对大量外部文本样本进行编码而得到的，它代表了模型中需要被保留的背景知识。其具体获取步骤如下：

- [cite_start]**数据来源**：研究人员首先从一个庞大的知识库中随机抽取大量的知识样本 [cite: 62, 63] [cite_start]。在论文的实际操作中，他们通常从维基百科 (Wikipedia) 中随机选择 100,000 个事实三元组 ((主语, 关系, 宾语)) [cite: 63]。
- **编码过程**：将每一个抽样出的知识三元组转换成相应的自然语言提示 (prompt)，然后将这个提示输入到语言模型中。
- [cite_start]**生成“键”向量**：当提示信息流经模型中被指定要编辑的前馈网络 (FFN) 层时，该层的输入激活值会被捕获并处理，从而为该知识样本生成一个对应的“键” (key) 向量 $\mathbf{k}$ [cite: 376]。
- [cite_start]**矩阵堆叠**：最后，将这 100,000 个独立的“键”向量 $\mathbf{k}$ 作为列向量横向拼接在一起，最终构成了高维矩阵 K_0 [cite: 63, 379]。

投影矩阵 P 是通过以下三个步骤得到的：

- [cite_start]**计算协方差矩阵**：首先，为了降低计算的复杂度，研究人员没有直接使用高维的“需保留知识”键矩阵 K_0 ，而是计算了其非中心协方差矩阵 $K_0 K_0^T$ [cite: 78][cite_start]。这个矩阵的维度要小得多，使得后续计算变得可行 [cite: 78]。
- [cite_start]**进行奇异值分解 (SVD)**：对协方差矩阵 $K_0 K_0^T$ 进行奇异值分解 [cite: 79]。 $\{U, \Lambda, U^T\} = SVD(K_0 K_0^T)$
[cite_start]分解后得到的矩阵 U 由 $K_0 K_0^T$ 的所有特征向量组成 [cite: 79]。
- 筛选零空间特征向量并构建矩阵**：
 - [cite_start]从矩阵 U 中筛选出与零（或接近零）特征值对应的特征向量 [cite: 79] [cite_start]。在实际应用中，由于数值计算的误差，特征值很少会严格等于零，因此论文设定了一个阈值（如 10^{-2} ），并移除所有对应特征值高于此阈值的特征向量 [cite: 84]。
 - [cite_start]将筛选后剩余的（即对应零特征值的）特征向量构成一个新的子矩阵，记为 $\hat{U}$ [cite: 80]。
 - [cite_start]最终，投影矩阵 P 通过将子矩阵 $\hat{U}$ 与其自身的转置 $(\hat{U})^T$ 相乘得到 [cite: 80]：
$$P = \hat{U}(\hat{U})^T$$

[cite_start]这个构建出的矩阵 P 能够将任意扰动向量映射到 K_0 的零空间上，从而保证编辑过程不会影响到“需保留知识” [cite: 80, 81]。

Δ 的求解

[cite_start]根据您提供的论文，扰动 Δ 并不是通过迭代训练（如梯度下降）来求解的，而是通过求解一个数学方程直接计算出来的，这被称为**闭式解 (closed-form solution)** [cite: 98, 381]。

[cite_start]求解的具体方法是使用线性代数中的**正规方程 (normal equation)** [cite: 97]。整个求解过程如下：

- 确立优化目标**：首先，写出需要最小化的目标函数 J 。在序贯编辑的场景下，这个目标函数是：
[cite_start] $J = \|(W + \tilde{\Delta}P)K_1 - V_1\|^2 + \|\tilde{\Delta}P\|^2 + \|\tilde{\Delta}PK_p\|^2$ [cite: 96, 393]
- [cite_start]**求导并设为零**：为了找到能让目标函数 J 最小的 $\tilde{\Delta}$ ，需要对 J 关于 $\tilde{\Delta}$ 求矩阵导数，并令其结果等于零 [cite: 393]。这一步得出的方程是：
[cite_start] $(\Delta PK_1 - R)K_1^T P + \Delta P + \Delta PK_p K_p^T P = 0$ [cite: 97]
 - [cite_start]这里的 R 是残差，定义为 $R = V_1 - WK_1$ [cite: 97, 393]。
- [cite_start]**求解方程**：最后，通过代数变换解出上述方程，即可得到最终的扰动矩阵 $\Delta_{AlphaEdit}$ [cite: 98]。

最终的解为：

$$[cite_start]\Delta_{AlphaEdit} = RK_1^T P(K_p K_p^T P + K_1 K_1^T P + I)^{-1} [cite: 98]$$

[cite_start]这个公式就是计算最终扰动 Δ 的直接方法。公式中的每一个组成部分（如 R, K_1, K_p, P, I ）都是已知的或可以计算的，因此可以直接代入并计算出最终结果，然后将其加到原始模型权重上。论文还证明了公式中需要求逆的矩阵项 $(K_p K_p^T P + K_1 K_1^T P + I)$ 是正定的，因此保证了其总是可逆的，使得解一定存在 [cite: 99, 395, 396, 397]。